

close to the eye point. A w-based depth buffer isn't perfect, and can sometimes exhibit hidden surface artifacts for near objects. Another drawback to the w-buffered approach is related to hardware support: w-buffering isn't supported as widely in hardware as z-buffering.

Using a z-buffer requires overhead during rendering. Various techniques can be used to optimize rendering when using z-buffers. Applications can increase performance when using z-buffering and texturing by ensuring that scenes are rendered from front to back. Textured z-buffered primitives are pretested against the z-buffer on a scan line basis. If a scan line is hidden by a previously rendered polygon, the system rejects it quickly and efficiently. Z-buffering can improve performance, but the technique is most useful when a scene draws the same pixels more than once. This is difficult to calculate exactly, but you can often make a close approximation. If the same pixels are drawn less than twice, you can achieve the best performance by turning z-buffering off and rendering the scene from back to front.

The actual interpretation of a depth value is specific to the renderer.

Related topics

[Depth and stencil buffers](#)

Stencil buffers

Article • 10/20/2022

A *stencil buffer* is used to mask pixels in an image, to produce special effects. The mask controls whether the pixel is drawn or not. These special effects include compositing; decaling; dissolves, fades, and swipes; outlines and silhouettes; and two-sided stencil. Some of the more common effects are shown below.

The stencil buffer enables or disables drawing to the rendering target surface on a pixel-by-pixel basis. At its most fundamental level, it enables applications to mask sections of the rendered image so that they are not displayed. Applications often use stencil buffers for special effects such as dissolves, decaling, and outlining.

Stencil buffer information is embedded in the z-buffer data.

How the stencil buffer works

Direct3D performs a test on the contents of the stencil buffer on a pixel-by-pixel basis. For each pixel in the target surface, it performs a test using the corresponding value in the stencil buffer, a stencil reference value, and a stencil mask value. If the test passes, Direct3D performs an action. The test is performed using the following steps.

1. Perform a bitwise AND operation of the stencil reference value with the stencil mask.
2. Perform a bitwise AND operation of the stencil buffer value for the current pixel with the stencil mask.
3. Compare the result of step 1 to the result of step 2, using the comparison function.

The above steps are shown in the following line of code:

```
C++
```

```
(StencilRef & StencilMask) CompFunc (StencilBufferValue & StencilMask)
```

- StencilRef represents the stencil reference value.
- StencilMask represents the value of the stencil mask.
- CompFunc is the comparison function.
- StencilBufferValue is the contents of the stencil buffer for the current pixel.
- The ampersand (&) symbol represents the bitwise AND operation.

The current pixel is written to the target surface if the stencil test passes, and is ignored otherwise. The default comparison behavior is to write the pixel, no matter how each bitwise operation turns out. You can change this behavior by changing the value of an enumerated type to identify the desired comparison function.

Your application can customize the operation of the stencil buffer. It can set the comparison function, the stencil mask, and the stencil reference value. It can also control the action that Direct3D takes when the stencil test passes or fails.

Compositing

Your application can use the stencil buffer to composite 2D or 3D images onto a 3D scene. A mask in the stencil buffer is used to occlude an area of the rendering target surface. Stored 2D information, such as text or bitmaps, can then be written to the occluded area. Alternately, your application can render additional 3D primitives to the stencil-masked region of the rendering target surface. It can even render an entire scene.

Games often composite multiple 3D scenes together. For instance, driving games typically display a rear-view mirror. The mirror contains the view of the 3D scene behind the driver. It is essentially a second 3D scene composited with the driver's forward view.

Decaling

Direct3D applications use decaling to control which pixels from a particular primitive image are drawn to the rendering target surface. Applications apply decals to the images of primitives to enable coplanar polygons to render correctly.

For instance, when applying tire marks and yellow lines to a roadway, the markings should appear directly on top of the road. However, the z values of the markings and the road are the same. Therefore, the depth buffer might not produce a clean separation between the two. Some pixels in the back primitive may be rendered on top of the front primitive and vice versa. The resulting image appears to shimmer from frame to frame. This effect is called *z-fighting* or *flimmering*.

To solve this problem, use a stencil to mask the section of the back primitive where the decal will appear. Turn off z-buffering and render the image of the front primitive into the masked-off area of the render-target surface.

Although multiple texture blending can be used to solve this problem, doing so limits the number of other special effects that your application can produce. Using the stencil buffer to apply decals frees up texture blending stages for other effects.

Dissolves, fades, and swipes

Increasingly, applications employ special effects that are commonly used in movies and video, such as dissolves, swipes, and fades.

In a dissolve, one image is gradually replaced by another in a smooth sequence of frames. Although Direct3D provides methods of using multiple texture blending to achieve the same effect, applications that use the stencil buffer for dissolves can use texture-blending capabilities for other effects while they do a dissolve.

When your application performs a dissolve, it must render two different images. It uses the stencil buffer to control which pixels from each image are drawn to the rendering target surface. You can define a series of stencil masks and copy them into the stencil buffer on successive frames. Alternately, you can define a base stencil mask for the first frame and alter it incrementally.

At the beginning of the dissolve, your application sets the stencil function and stencil mask so that most of the pixels from the starting image pass the stencil test. Most of the pixels from the ending image should fail the stencil test. On successive frames, the stencil mask is updated so that fewer and fewer of the pixels in the starting image pass the test. As the frames progress, fewer and fewer of the pixels in the ending image fail the test. In this manner, your application can perform a dissolve using any arbitrary dissolve pattern.

Fading in or fading out is a special case of dissolving. When fading in, the stencil buffer is used to dissolve from a black or white image to a rendering of a 3D scene. Fading out is the opposite, your application starts with a rendering of a 3D scene and dissolves to black or white. The fade can be done using any arbitrary pattern you want to employ.

Direct3D applications use a similar technique for swipes. For example, when an application performs a left-to-right swipe, the ending image appears to slide gradually on top of the starting image from left to right. As in a dissolve, you must define a series of stencil masks that are loaded into the stencil buffer on successive frames, or successively modify the starting stencil mask. The stencil masks are used to disable the writing of pixels from the starting image and to enable the writing of pixels from the ending image.

A swipe is somewhat more complex than a dissolve in that your application must read pixels from the ending image in the reverse order of the swipe. That is, if the swipe is moving from left to right, your application must read pixels from the ending image from right to left.

Outlines and silhouettes

You can use the stencil buffer for more abstract effects, such as outlining and silhouetting.

If your application applies a stencil mask to the image of a primitive that is the same shape but slightly smaller, the resulting image contains only the primitive's outline. The application can then fill the stencil-masked area of the image with a solid color, giving the primitive an embossed look.

If the stencil mask is the same size and shape as the primitive you are rendering, the resulting image contains a hole where the primitive should be. Your application can then fill the hole with black to produce a silhouette of the primitive.

Two-sided stencil

Shadow Volumes are used for drawing shadows with the stencil buffer. The application computes the shadow volumes cast by occluding geometry, by computing the silhouette edges and extruding them away from the light into a set of 3D volumes. These volumes are then rendered twice into the stencil buffer.

The first render draws forward-facing polygons, and increments the stencil-buffer values. The second render draws the back-facing polygons of the shadow volume, and decrements the stencil buffer values. Normally, all incremented and decremented values cancel each other out. However, the scene was already rendered with normal geometry causing some pixels to fail the z-buffer test as the shadow volume is rendered. Values left in the stencil buffer correspond to pixels that are in the shadow. These remaining stencil-buffer contents are used as a mask, to alpha-blend a large, all-encompassing black quad into the scene. With the stencil buffer acting as a mask, the result is to darken pixels that are in the shadows.

This means that the shadow geometry is drawn twice per light source, hence putting pressure on the vertex throughput of the GPU. The two-sided stencil feature has been designed to mitigate this situation. In this approach, there are two sets of stencil state (named below), one set each for the front-facing triangles and the other for the back-facing triangles. This way, only a single pass is drawn per shadow volume, per light.

Related topics

[Depth and stencil buffers](#)

Textures

Article • 10/20/2022

Textures are a powerful tool in creating realism in computer-generated 3D images. Direct3D supports an extensive texturing feature set, providing developers with easy access to advanced texturing techniques.

For improved performance, consider using dynamic textures. A dynamic texture can be locked, written to, and unlocked each frame.

In this section

Topic	Description
Introduction to textures	A texture resource is a data structure to store texels, which are the smallest unit of a texture that can be read or written to. When the texture is read by a shader, it can be filtered by texture samplers.
Basic texturing concepts	Early computer-generated 3D images, although generally advanced for their time, tended to have a shiny plastic look. They lacked the types of markings-such as scuffs, cracks, fingerprints, and smudges-that give 3D objects realistic visual complexity. Textures have become popular for enhancing the realism of computer-generated 3D images.
Texture addressing modes	Your Direct3D application can assign texture coordinates to any vertex of any primitive. Typically, the u- and v-texture coordinates that you assign to a vertex are in the range of 0.0 to 1.0 inclusive. However, by assigning texture coordinates outside that range, you can create certain special texturing effects.
Texture filtering	Texture filtering produces a color for each pixel in the primitive's 2D rendered image when a primitive is rendered by mapping a 3D primitive onto a 2D screen.
Texture resources	Textures are a type of resource used for rendering.

Topic	Description
Texture wrapping	Texture wrapping changes the basic way that Direct3D rasterizes textured polygons using the texture coordinates specified for each vertex. While rasterizing a polygon, the system interpolates between the texture coordinates at each of the polygon's vertices to determine the texels that should be used for every pixel of the polygon.
Texture blending	Direct3D can blend as many as eight textures onto primitives in a single pass. The use of multiple texture blending can profoundly increase the frame rate of a Direct3D application. An application employs multiple texture blending to apply textures, shadows, specular lighting, diffuse lighting, and other special effects in a single pass.
Light mapping with textures	A light map is a texture or group of textures that contains information about lighting in a 3D scene. Light maps map areas of light and shadow onto primitives. Multipass and multiple texture blending enable your application to render scenes with a more realistic appearance than shading techniques.
Compressed texture resources	Texture maps are digitized images drawn on three-dimensional shapes to add visual detail. They are mapped into these shapes during rasterization, and the process can consume large amounts of both the system bus and memory. To reduce the amount of memory consumed by textures, Direct3D supports the compression of texture surfaces. Some Direct3D devices support compressed texture surfaces natively.

Related topics

[Direct3D Graphics Learning Guide](#)

Introduction to textures

Article • 10/20/2022

A texture resource is a data structure to store texels, which are the smallest unit of a texture that can be read or written to. When the texture is read by a shader, it can be filtered by texture samplers.

A texture resource is a structured collection of data designed to store texels. A texel represents the smallest unit of a texture that can be read or written to by the pipeline. Unlike buffers, textures can be filtered by texture samplers as they are read by shader units. The type of texture impacts how the texture is filtered. Each texel contains 1 to 4 components, arranged in one of the DXGI formats defined by the DXGI_FORMAT enumeration.

Textures are created as a structured resource with a known size. However, each texture may be typed or typeless when the resource is created as long as the type is fully specified using a view when the texture is bound to the pipeline.

Texture Types

Direct3D supports several floating-point representations. All floating-point computations operate under a defined subset of the IEEE 754 32-bit single precision floating-point rules.

There are several types of textures: 1D, 2D, 3D, each of which can be created with or without mipmaps. Direct3D also supports texture arrays and multisampled textures.

- [1D Textures](#)
- [1D Texture Arrays](#)
- [2D Textures and 2D Texture Arrays](#)
- [3D Textures](#)

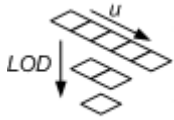
1D Textures

A 1D texture in its simplest form contains texture data that can be addressed with a single texture coordinate; it can be visualized as an array of texels, as shown in the following illustration.

The following illustration shows a 1D texture:



Each texel contains a number of color components depending on the format of the data stored. Adding more complexity, you can create a 1D texture with mipmap levels, as shown in the following illustration.

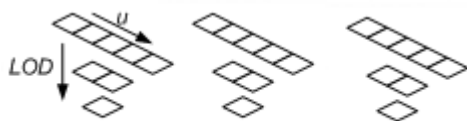


A mipmap level is a texture that is a power-of-two smaller than the level above it. The topmost level contains the most detail, each subsequent level is smaller. For a 1D mipmap, the smallest level contains one texel. Furthermore, MIP levels always reduce down to 1:1.

When mipmaps are generated for an odd sized texture, the next lower level is always even size (except when the lowest level reaches 1). For example, the diagram illustrates a 5x1 texture whose next lowest level is a 2x1 texture, whose next (and last) mip level is a 1x1 sized texture. The levels are identified by an index called a LOD (level-of-detail) which is used to access the smaller texture when rendering geometry that is not as close to the camera.

1D Texture Arrays

Direct3D also supports arrays of textures. An array of 1D textures looks conceptually like the following illustration.



This texture array contains three textures. Each of the three textures has a texture width of 5 (which is the number of elements in the first layer). Each texture also contains a 3 layer mipmap.

All texture arrays in Direct3D are a homogeneous array of textures; this means that every texture in a texture array must have the same data format and size (including texture width and number of mipmap levels). You may create texture arrays of different sizes, as long as all the textures in each array match in size.

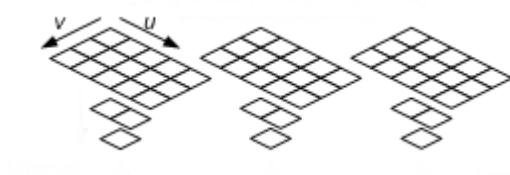
2D Textures and 2D Texture Arrays

A Texture2D resource contains a 2D grid of texels. Each texel is addressable by a u, v vector. Since it is a texture resource, it may contain mipmap levels, and subresources. A fully populated 2D texture resource looks like the following illustration.



This texture resource contains a single 3x5 texture with three mipmap levels.

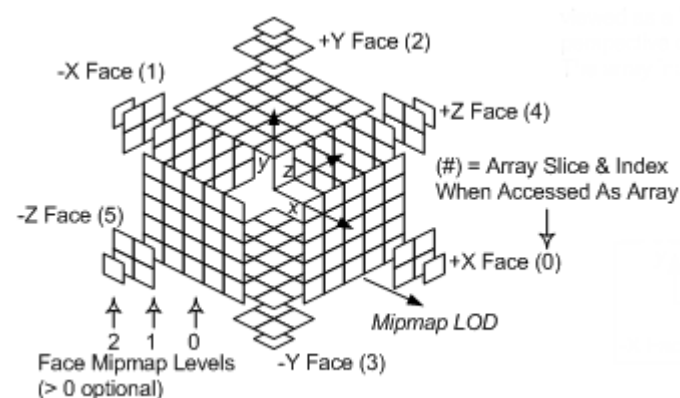
A 2D texture array resource is a homogeneous array of 2D textures; that is, each texture has the same data format and dimensions (including mipmap levels). It has a similar layout as the 1D texture array except that the textures now contain 2D data, as shown in the following illustration.



This texture array contains three textures; each texture is 3x5 with two mipmap levels.

Using a 2D Texture Array as a Texture Cube

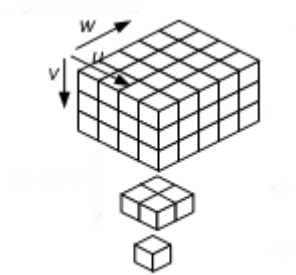
A texture cube is a 2D texture array that contains 6 textures, one for each face of the cube. A fully populated texture cube looks like the following illustration.



A 2D texture array that contains 6 textures may be read from within shaders with the cube map intrinsic functions, after they are bound to the pipeline with a cube-texture view. Texture cubes are addressed from the shader with a 3D vector pointing out from the center of the texture cube.

3D Textures

A 3D texture resource (also known as a volume texture) contains a 3D volume of texels. Because it is a texture resource, it may contain mipmap levels. A fully populated 3D texture looks like the following illustration.



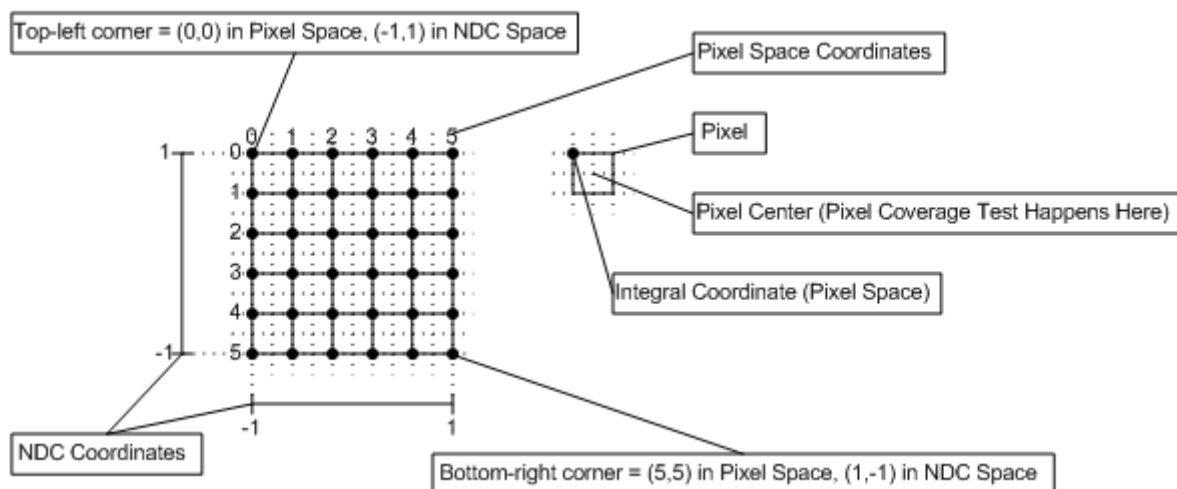
When a 3D texture mipmap slice is bound as a render target output (with a render-target view), the 3D texture behaves identically to a 2D texture array with n slices. The particular render slice is chosen from the geometry-shader stage.

There is no concept of a 3D texture array; therefore a 3D texture subresource is a single mipmap level.

Coordinate systems for Direct3D are defined for pixels and texels.

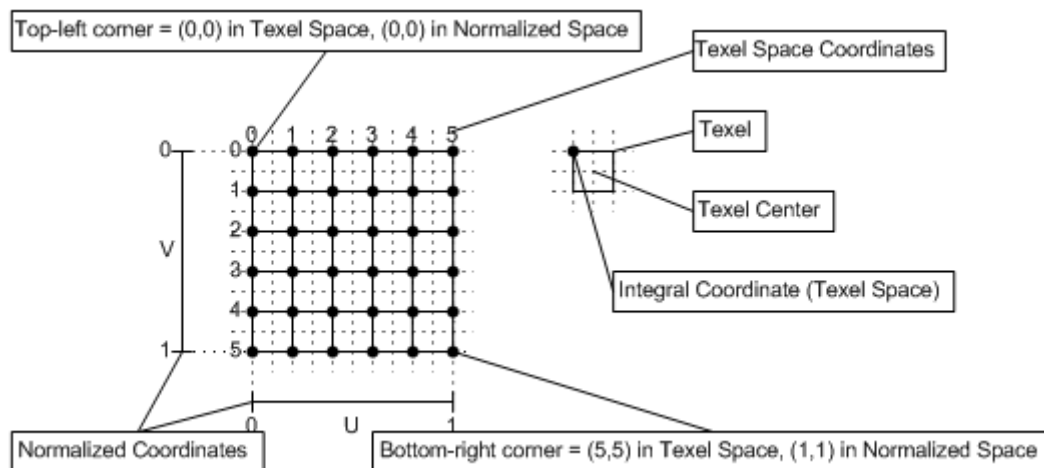
Pixel Coordinate System

The pixel coordinate system in Direct3D defines the origin of a render target at the upper-left corner, as shown in the following diagram. Pixel centers are offset by $(0.5f, 0.5f)$ from integer locations.



Texel Coordinate System

The texel coordinate system has its origin at the top-left corner of the texture, as shown in the following diagram. This makes rendering screen-aligned textures trivial, as the pixel coordinate system is aligned with the texel coordinate system.



Texture coordinates are represented with either a normalized or a scaled number; each texture coordinate is mapped to a specific texel as follows:

For a normalized coordinate:

- Point sampling: $\text{Texel \#} = \text{floor}(U * \text{Width})$
- Linear sampling: $\text{Left Texel \#} = \text{floor}(U * \text{Width})$, $\text{Right Texel \#} = \text{Left Texel \#} + 1$

For a scaled coordinate:

- Point sampling: $\text{Texel \#} = \text{floor}(U)$
- Linear sampling: $\text{Left Texel \#} = \text{floor}(U - 0.5)$, $\text{Right Texel \#} = \text{Left Texel \#} + 1$

Where the width, is the width of the texture (in texels).

Texture address wrapping occurs after the texel location is computed.

Related topics

[Textures](#)

Basic texturing concepts

Article • 10/20/2022

Early computer-generated 3D images, although generally advanced for their time, tended to have a shiny plastic look. They lacked the types of markings-such as scuffs, cracks, fingerprints, and smudges-that give 3D objects realistic visual complexity. Textures have become popular for enhancing the realism of computer-generated 3D images.

In its everyday use, the word texture refers to an object's smoothness or roughness. In computer graphics, however, a texture is a bitmap of pixel colors that give an object the appearance of texture.

Because Direct3D textures are bitmaps, any bitmap can be applied to a Direct3D primitive. For instance, applications can create and manipulate objects that appear to have a wood grain pattern in them. Grass, dirt, and rocks can be applied to a set of 3D primitives that form a hill. The result is a realistic-looking hillside. You can also use texturing to create effects such as signs along a roadside, rock strata in a cliff, or the appearance of marble on a floor.

In addition, Direct3D supports more advanced texturing techniques such as texture blending-with or without transparency-and light mapping. See [Texture blending](#) and [Light mapping with textures](#).

If your application creates a hardware abstraction layer (HAL) device or a software device, it can use 8, 16, 24, 32, 64, or 128-bit textures.

Related topics

[Textures](#)

Texture addressing modes

Article • 10/20/2022

Your Direct3D application can assign texture coordinates to any vertex of any primitive. Typically, the u- and v-texture coordinates that you assign to a vertex are in the range of 0.0 to 1.0 inclusive. However, by assigning texture coordinates outside that range, you can create certain special texturing effects. .

You control what Direct3D does with texture coordinates that are outside the [0.0, 1.0] range by setting the texture addressing mode. For instance, you can have your application set the texture addressing mode so that a texture is tiled across a primitive.

Direct3D enables applications to perform texture wrapping. See [Texture wrapping](#).

Enabling texture wrapping effectively makes texture coordinates outside the [0.0, 1.0] range invalid, and the behavior for rasterizing such delinquent texture coordinates is undefined in this case. When texture wrapping is enabled, texture addressing modes are not used. Take care that your application does not specify texture coordinates lower than 0.0 or higher than 1.0 when texture wrapping is enabled.

Summary of the texture addressing modes

Texture addressing mode	Description
Wrap	Repeats the texture on every integer junction.
Mirror	Mirrors the texture at every integer boundary.
Clamp	Clamps your texture coordinates to the [0.0, 1.0] range; Clamp mode applies the texture once, then smears the color of edge pixels.
Border Color	Uses an arbitrary <i>border color</i> for any texture coordinates outside the range of 0.0 through 1.0, inclusive.

Wrap texture address mode

The Wrap texture address mode makes Direct3D repeat the texture on every integer junction.

Suppose, for example, your application creates a square primitive and specifies texture coordinates of (0.0,0.0), (0.0,3.0), (3.0,3.0), and (3.0,0.0). Setting the texture addressing mode to "Wrap" results in the texture being applied three times in both the u-and v-directions, as shown in the following illustration.



Contrast this with the following **Mirror** texture address mode.

Mirror texture address mode

The Mirror texture address mode causes Direct3D to mirror the texture at every integer boundary.

Suppose, for example, your application creates a square primitive and specifies texture coordinates of (0.0,0.0), (0.0,3.0), (3.0,3.0), and (3.0,0.0). Setting the texture addressing mode to "Mirror" results in the texture being applied three times in both the u- and v-directions. Every other row and column that it is applied to is a mirror image of the preceding row or column, as shown in the following illustration.



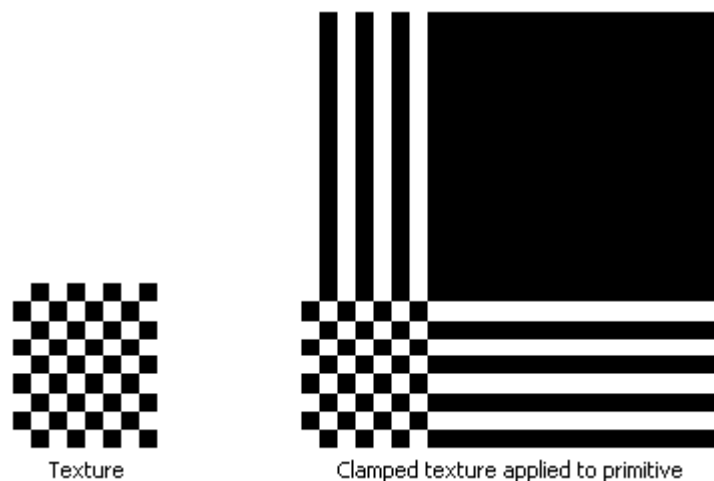
Contrast this with the previous **Wrap texture address mode**.

Clamp texture address mode

The Clamp texture address mode causes Direct3D to clamp your texture coordinates to the [0.0, 1.0] range; Clamp mode applies the texture once, then smears the color of edge pixels.

Suppose that your application creates a square primitive and assigns texture coordinates of (0.0,0.0), (0.0,3.0), (3.0,3.0), and (3.0,0.0) to the primitive's vertices. Setting the texture addressing mode to "Clamp" results in the texture being applied once. The pixel colors at the top of the columns and the end of the rows are extended to the top and right of the primitive respectively.

The following illustration shows a clamped texture.



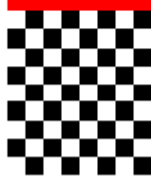
Border Color texture address mode

The Border Color texture address mode causes Direct3D to use an arbitrary color, known as the *border color*, for any texture coordinates outside the range of 0.0 through 1.0, inclusive.

In the following illustration, the application specifies that the texture be applied to the primitive using a red border.



Texture



Texture with red border applied to primitive

Device limitations

Although the system generally allows texture coordinates outside the range of 0.0 and 1.0, inclusive, hardware limitations often affect how far outside that range texture coordinates can be. A rendering device communicates the limit for the full range of texture coordinates allowed by the device, when you retrieve the device's capabilities.

For instance, if this value is 128, then the input texture coordinates must be kept in the range -128.0 to +128.0. Passing vertices with texture coordinates outside this range is invalid. The same restriction applies to the texture coordinates generated as a result of automatic texture coordinate generation and texture coordinate transformations.

Texture repeating limitations can depend on the size of the texture indexed by the texture coordinates. In that case, supposing the texture dimension is 32, and the range of texture coordinates allowed by the device is 512, the actual valid texture coordinate range would therefore be $512/32 = 16$, so the texture coordinates for this device must be within the range of -16.0 to +16.0.

Related topics

[Textures](#)

Texture filtering

Article • 10/20/2022

Texture filtering produces a color for each pixel in the primitive's 2D rendered image when a primitive is rendered by mapping a 3D primitive onto a 2D screen.

When Direct3D renders a primitive, it maps the 3D primitive onto a 2D screen. If the primitive has a texture, Direct3D must use that texture to produce a color for each pixel in the primitive's 2D rendered image. For every pixel in the primitive's on-screen image, it must obtain a color value from the texture. This process is called *texture filtering*.

When a texture filter operation is performed, the texture being used is typically also being magnified or minified. In other words, it is being mapped into a primitive image that is larger or smaller than itself. Magnification of a texture can result in many pixels being mapped to one texel. The result can be a chunky appearance. Minification of a texture often means that a single pixel is mapped to many texels. The resulting image can be blurry or aliased. To resolve these problems, some blending of the texel colors must be performed to arrive at a color for the pixel.

Direct3D simplifies the complex process of texture filtering. It provides you with three types of texture filtering - linear filtering, anisotropic filtering, and mipmap filtering. If you select no texture filtering, Direct3D uses a technique called nearest-point sampling.

Each type of texture filtering has advantages and disadvantages. For instance, linear texture filtering can produce jagged edges or a chunky appearance in the final image. However, it is a computationally low-overhead method of texture filtering. Filtering with mipmaps usually produces the best results, especially when combined with anisotropic filtering. However, it requires the most memory of the techniques that Direct3D supports.

Types of texture filtering

Direct3D supports the following texture filtering approaches.

In this section

Topic	Description
-------	-------------

Topic	Description
Nearest-point sampling	Applications are not required to use texture filtering. Direct3D can be set so that it computes the texel address, which often does not evaluate to integers, and copies the color of the texel with the closest integer address. This process is called <i>nearest-point sampling</i> .
Bilinear texture filtering	<i>Bilinear filtering</i> calculates the weighted average of the 4 texels closest to the sampling point. This filtering approach is more accurate and common than nearest-point filtering. This approach is efficient because it is implemented in modern graphics hardware.
Anisotropic texture filtering	<i>Anisotropy</i> is the distortion visible in the texels of a 3D object whose surface is oriented at an angle with respect to the plane of the screen. When a pixel from an anisotropic primitive is mapped to texels, its shape is distorted.
Texture filtering with mipmaps	A <i>mipmap</i> is a sequence of textures, each of which is a progressively lower resolution representation of the same image. The height and width of each image, or level, in the mipmap is a power-of-two smaller than the previous level.

Related topics

[Textures](#)

Nearest-point sampling

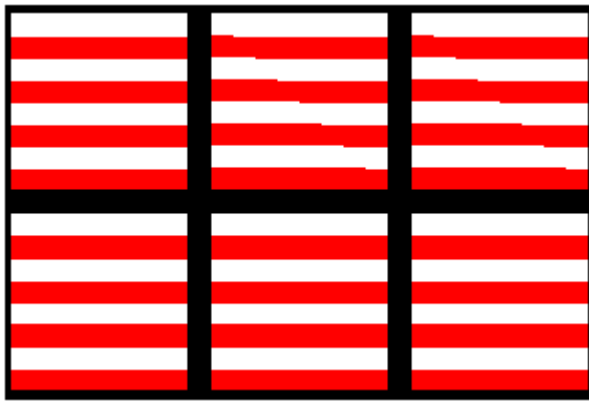
Article • 10/20/2022

Applications are not required to use texture filtering. Direct3D can be set so that it computes the texel address, which often does not evaluate to integers, and copies the color of the texel with the closest integer address. This process is called *nearest-point sampling*. Nearest-point sampling can be as if the size of the texture is similar to the size of the primitive's image on the screen. If not, the texture must be magnified or minified. The result of mismatching texture sizes to primitive image size can be a chunky, aliased, or blurred image.

Use nearest-point sampling carefully, because it can sometimes cause graphic artifacts when the texture is sampled at the boundary between two texels. This boundary is the position along the texture (u or v) at which the sampled texel transitions from one texel to the next. When point sampling is used, the system chooses one sample texel or the other, and the result can change abruptly from one texel to the next texel as the boundary is crossed. This effect can appear as undesired graphic artifacts in the displayed texture. When linear filtering is used, the resulting texel is computed from both adjacent texels and smoothly blends between them as the texture index moves through the boundary.

This effect can be seen when mapping a very small texture onto a very large polygon: an operation often called magnification. For example, when using a texture that looks like a checkerboard, nearest-point sampling results in a larger checkerboard that shows distinct edges. By contrast, linear texture filtering results in an image where the checkerboard colors vary smoothly across the polygon.

In most cases, applications receive the best results by avoiding nearest-point sample wherever possible. The majority of hardware today is optimized for linear filtering, so your application should not suffer degraded performance. If the effect you desire absolutely requires the use of the nearest-point sampling - such as when using textures to display readable text characters - then your application should be extremely careful to avoid sampling at the texel boundaries, which might result in undesired effects. The following illustration shows what these artifacts can look like.



The two squares in the upper right of the group appear different than their neighbors, with diagonal offsets running through them. To avoid graphic artifacts like these, you must be familiar with Direct3D texture sampling rules for nearest-point filtering. Direct3D maps a floating-point texture coordinate ranging from $[0.0, 1.0]$ (0.0 to 1.0, inclusive) to an integer texel space value ranging from $[-0.5, n - 0.5]$, where n is the number of texels in a given dimension on the texture. The resulting texture index is rounded to the nearest integer. This mapping can introduce sampling inaccuracies at texel boundaries.

For a simple example, imagine an application that renders polygons with the Wrap texture addressing mode. Using the mapping employed by Direct3D, the u texture index maps as show in the following diagram for a texture with a width of 4 texels.



The texture coordinates, 0.0 and 1.0 for this illustration, are exactly at the boundary between texels. Using the method by which Direct3D maps values, the texture coordinates range from $[-0.5, 4 - 0.5]$, where 4 is the width of the texture. For this case, the sampled texel is the 0 texel for a texture index of 1.0. However, if the texture coordinate was only slightly less than 1.0, the sampled texel would be the n texel instead of the 0 texel.

The implication of this is that magnifying a small texture using texture coordinates of exactly 0.0 and 1.0 with nearest-point filtering on a screen-space aligned triangle results in pixels for which the texture map is sampled at the boundary between texels. Any inaccuracies in the computation of texture coordinates, however small, results in artifacts along the areas in the rendered image which correspond to the texel edges of the texture map.

Performing this mapping of floating point texture coordinates to integer texels with perfect accuracy is difficult, computationally time-consuming, and generally not necessary. Most hardware implementations use an iterative approach for computing texture coordinates at each pixel location within a triangle. Iterative approaches tend to hide these inaccuracies because the errors are accumulated evenly during iteration.

The Direct3D reference rasterizer uses a direct-evaluation approach for computing texture indexes at each pixel location. Direct evaluation differs from the iterative approach in that any inaccuracy in the operation exhibits a more random error distribution. The result of this is that the sampling errors that occur at the boundaries can be more noticeable because the reference rasterizer does not perform this operation with perfect accuracy.

The best approach is to use nearest-point filtering only when necessary. When you must use it, it is recommended that you offset texture coordinates slightly from the boundary positions to avoid artifacts.

Related topics

[Texture filtering](#)

Bilinear texture filtering

Article • 10/20/2022

Bilinear filtering calculates the weighted average of the 4 texels closest to the sampling point. This filtering approach is more accurate and common than nearest-point filtering. This approach is efficient because it is implemented in modern graphics hardware.

Example

Textures are always linearly addressed from (0.0, 0.0) at the top-left corner to (1.0, 1.0) at the bottom-right corner. Linear addressing of a texture is shown in the following illustration.

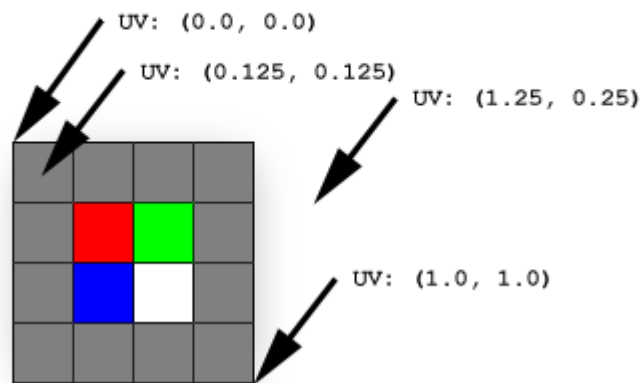


Figure 7a

Textures are linearly addressed from (0, 0) to (1, 1). The address at the center of the top-left texel in a 4x4 texture is therefore (0.125, 0.125). Note that points outside the texture can also be addressed in the same space.

Textures are usually represented as if they were composed of solid blocks of color, but it's actually more correct to think of textures the same way you should think of the raster display: Each texel is defined at the exact center of a grid cell, as shown in the following illustration.

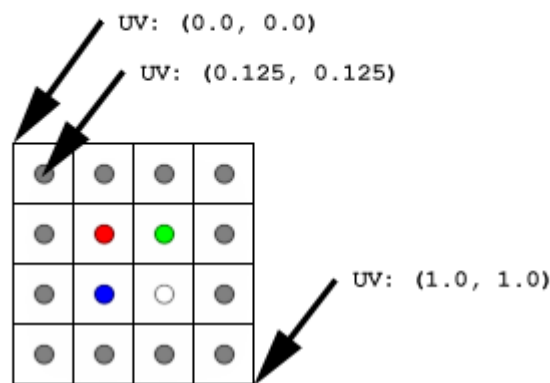


Figure 7b

Texels can be thought of as off-screen equivalents to pixels, where each texel is defined at the exact center of a grid cell.

If you ask the texture sampler for this texture's color at UV coordinates (0.375, 0.375) you'll get solid red (255, 0, 0). That makes sense because the center of the red texel cell is at UV (0.375, 0.375). What if you ask the sampler for the texture's color at UV (0.25, 0.25)? That's not as easy, because the point at UV (0.25, 0.25) lies at the exact corner of 4 texels.

The simplest scheme is simply to have the sampler return the color of the closest texel; this is called Point filtering (see [Nearest-point sampling](#)), and is usually undesirable due to grainy or blocky results. Point-sampling our texture at UV (0.25, 0.25) shows another subtle problem with nearest-point filtering: there are four texels equidistant from the sampling point, so there's no single nearest texel. One of those four texels will be chosen as the returned color, but the selection depends on how the coordinate is rounded, which may introduce tearing artifacts (see the Nearest-Point Sampling article in the SDK).

A slightly more accurate and more common filtering scheme is to calculate the weighted average of the 4 texels closest to the sampling point; this is called *Bilinear filtering*. The extra computational cost for Bilinear filtering is usually negligible because this routine is implemented in modern graphics hardware. Here are the colors we get at a few different sample points using bilinear filtering:

C++

UV: (0.5, 0.5)

This point is at the exact border between red, green, blue, and white texels. The color the sampler returns is gray:

C++

```
0.25 * (255, 0, 0)
0.25 * (0, 255, 0)
0.25 * (0, 0, 255)
## + 0.25 * (255, 255, 255)
-----
= (128, 128, 128)
```

C++

UV: (0.5, 0.375)

This point is at the midpoint of the border between red and green texels. The color the sampler returns is yellow-gray (note that the contributions of the blue and white texels are scaled to 0):

C++

```
0.5 * (255, 0, 0)
0.5 * (0, 255, 0)
0.0 * (0, 0, 255)
## + 0.0 * (255, 255, 255)
-----
= (128, 128, 0)
```

C++

UV: (0.375, 0.375)

This is the address of the red texel, which is the returned color (all other texels in the filtering calculation are weighted to 0):

C++

```
1.0 * (255, 0, 0)
0.0 * (0, 255, 0)
0.0 * (0, 0, 255)
## + 0.0 * (255, 255, 255)
-----
= (255, 0, 0)
```

Compare these calculations against the following illustration, which shows what happens if the bilinear filtering calculation is performed at every texture address across the 4x4 texture.

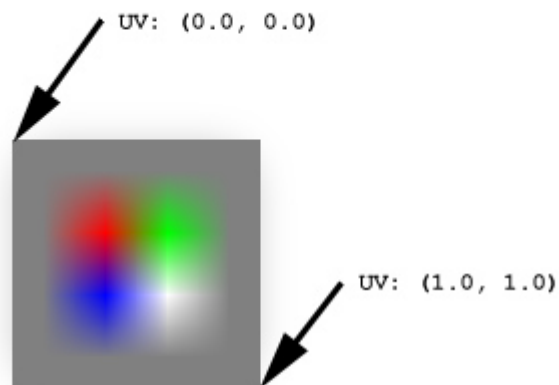


Figure 7c
This shows the color retrieved at each texture address when sampling the 4x4 texture using linear filtering.

Related topics

[Texture filtering](#)

Anisotropic texture filtering

Article • 10/20/2022

Anisotropy is the distortion visible in the texels of a 3D object whose surface is oriented at an angle with respect to the plane of the screen. When a pixel from an anisotropic primitive is mapped to texels, its shape is distorted. Direct3D measures the anisotropy of a pixel as the elongation - that is, length divided by width - of a screen pixel that is inverse-mapped into texture space.

You can use anisotropic texture filtering in conjunction with linear texture filtering or mipmap texture filtering to improve rendering results.

Related topics

[Texture filtering](#)

Texture filtering with mipmaps

Article • 10/20/2022

A *mipmap* is a sequence of textures, each of which is a progressively lower resolution representation of the same image. The height and width of each image, or level, in the mipmap is a power-of-two smaller than the previous level. Mipmaps do not have to be square.

A high-resolution mipmap image is used for objects that are close to the user. Lower-resolution images are used as the object appears farther away. Mipmapping improves the quality of rendered textures at the expense of using more memory.

Direct3D represents mipmaps as a chain of attached surfaces. The highest resolution texture is at the head of the chain and has the next level of the mipmap as an attachment. In turn, that level has an attachment that is the next level in the mipmap, and so on, down to the lowest resolution level of the mipmap.

The following illustrations shows an example of these levels. The bitmap textures represent a sign on a container in a 3D first-person game. When created as a mipmap, the highest resolution texture is first in the set. Each succeeding texture in the mipmap set is smaller in height and width by a power of 2. In this case, the maximum-resolution mipmap is 256 pixels by 256 pixels. The next, texture is 128x128. The last texture in the chain is 64x64.

This sign has a maximum distance from which it is visible. If the user begins far away from the sign, the game displays the smallest texture in the mipmap chain, which in this case the 64x64 texture.



As the user moves the point of view closer to the sign, progressively higher-resolution textures in the mipmap chain are used. The resolution in the following illustration is 128x128.



The highest resolution texture is used when the user's point of view is at the minimum allowable distance from the sign, as shown in the following illustration.

DANGER!

**Under no circumstances
should you open this container
without proper authorization. Do
not press this button.**



This is a more efficient way of simulating perspective for textures. Rather than render a single texture at many resolutions, it is faster to use multiple textures at varying resolutions.

Direct3D can assess which texture in a mipmap set is the closest resolution to the desired output, and it can map pixels into its texel space. If the resolution of the final image is between the resolutions of the textures in the mipmap set, Direct3D can examine texels in both mipmaps and blend their color values together.

To use mipmaps, your application must build a set of mipmaps. Applications apply mipmaps by selecting the mipmap set as the first texture in the set of current textures. See [Texture blending](#).

Next, your application must set the filtering method that Direct3D uses to sample texels. The fastest method of mipmap filtering is to have Direct3D select the nearest texel. Use the `D3DTEXF_POINT` enumerated value to select this. Direct3D can produce better filtering results if your application uses the `D3DTEXF_LINEAR` enumerated value. This selects the nearest mipmap, and then computes a weighted average of the texels surrounding the location in the texture to which the current pixel maps.

Mipmap textures are used in 3D scenes to decrease the time required to render a scene. They also improve the scene's realism. However, they often require large amounts of memory.

Note Each surface in a mipmap chain has dimensions that are one-half that of the previous surface in the chain. If the top-level mipmap has dimensions of 256x128, the dimensions of the second-level mipmap are 128x64, the third-level are 64x32, and so on, down to 1x1. You cannot request a number of mipmap levels in Levels that would cause either the width or height of any mipmap in the chain to be smaller than 1. In the

simple case of a 4x2 top-level mipmap surface, the maximum value allowed for Levels is three. The top-level dimensions are 4x2, the second-level dimensions are 2x1, and the dimensions for the third level are 1x1. A value larger than 3 in Levels results in a fractional value in the height of the second-level mipmap, and is therefore disallowed.

Direct3D can automatically perform mipmap texture filtering. Applications can manually traverse a mipmap chain to load bitmap data into each surface in the chain. This is often the only reason to traverse the chain. Automatically generating mipmaps at texture creation time takes advantage of hardware filtering because the mipmap resides in video memory.

Related topics

[Texture filtering](#)

Texture resources

Article • 10/20/2022

Textures are a type of resource used for rendering.

Rendering with texture resources

Direct3D supports multiple texture blending through the concept of texture stages. Each texture stage contains a texture and operations that can be performed on the texture.

The textures in the texture stages form the set of current textures. See [Texture blending](#).

The state of each texture is encapsulated in its texture stage.

Your application can also set the texture perspective and texture filtering states. See [Texture filtering](#).

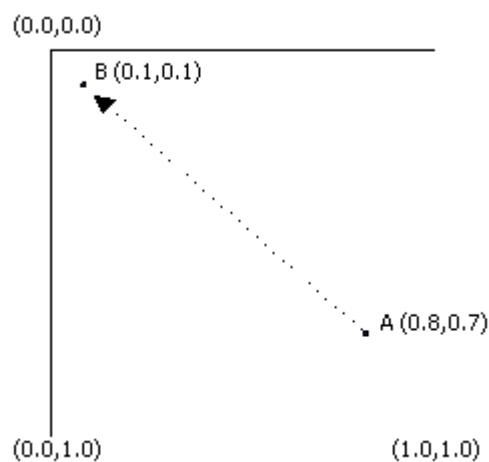
Related topics

[Textures](#)

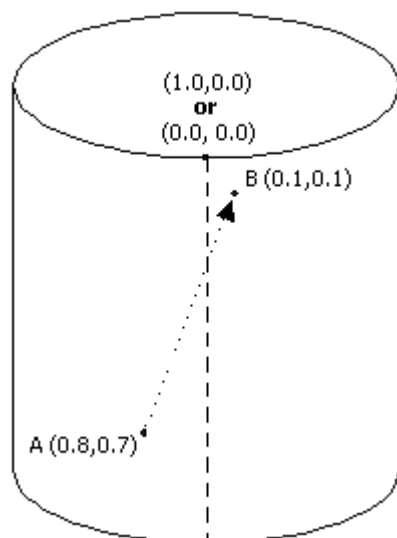
Texture wrapping

Article • 10/20/2022

Texture wrapping changes the basic way that Direct3D rasterizes textured polygons using the texture coordinates specified for each vertex. While rasterizing a polygon, the system interpolates between the texture coordinates at each of the polygon's vertices to determine the texels that should be used for every pixel of the polygon. Normally, the system treats the texture as a 2D plane, interpolating new texels by taking the shortest route from point A within a texture to point B. If point A represents the u, v position (0.8, 0.1), and point B is at (0.1, 0.1), the line of interpolation looks like the following diagram.

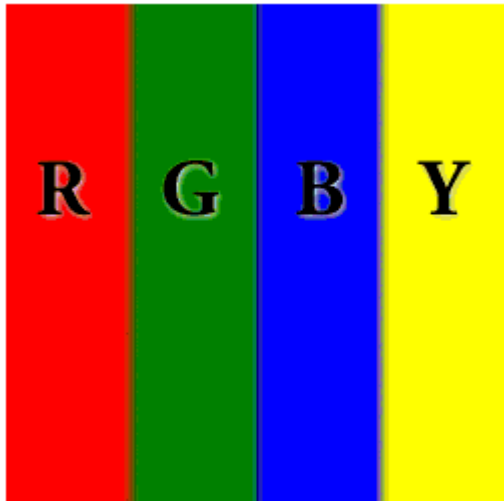


Note that the shortest distance between A and B in this illustration runs roughly through the middle of the texture. Enabling u-texture or v-texture coordinate wrapping changes how Direct3D perceives the shortest route between texture coordinates in the u-direction and v-direction. By definition, texture wrapping causes the rasterizer to take the shortest route between texture coordinate sets, assuming that 0.0 and 1.0 are coincident. The last bit is the tricky part: You can imagine that enabling texture wrapping in one direction causes the system to treat a texture as though it were wrapped around a cylinder. For example, consider the following diagram.

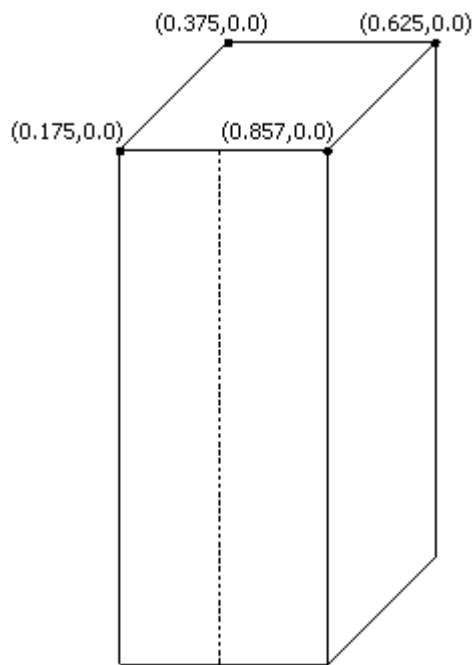


The preceding illustration shows how wrapping in the u - direction affects how the system interpolates texture coordinates. Using the same points as in the example for normal, or nonwrapped, textures, you can see that the shortest route between points A and B is no longer across the middle of the texture; it's now across the border where 0.0 and 1.0 exist together. Wrapping in the v-direction is similar, except that it wraps the texture around a cylinder that is lying on its side. Wrapping in both the u-direction and v-direction is more complex. In this situation, you can envision the texture as a torus, or doughnut.

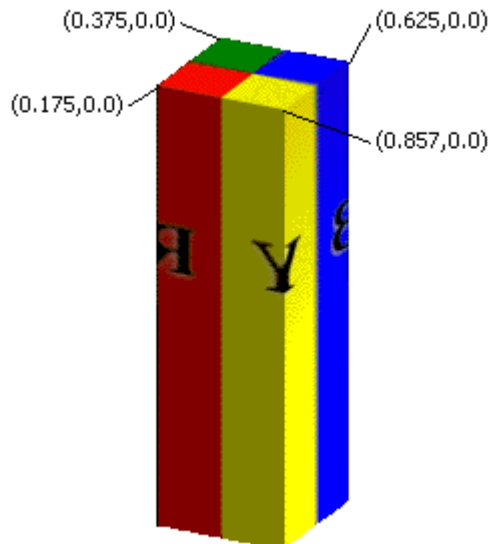
The most common practical application for texture wrapping is to perform environment mapping. Usually, an object textured with an environment map appears very reflective, showing a mirrored image of the object's surroundings in the scene. For the sake of this discussion, picture a room with four walls, each one painted with a letter R, G, B, Y and the corresponding colors: red, green, blue, and yellow. The environment map for such a simple room might look like the following illustration.



Imagine that the room's ceiling is held up by a perfectly reflective, four-sided, pillar. Mapping the environment map texture to the pillar is simple; making the pillar look as though it is reflecting the letters and colors is not as easy. The following diagram shows a wire frame of the pillar with the applicable texture coordinates listed near the top vertices. The seam where wrapping will cross the edges of the texture is shown with a dotted line.



With wrapping enabled in the u-direction, the textured pillar shows the colors and symbols from the environment map appropriately and, at the seam in the front of the texture, the rasterizer properly chooses the shortest route between the texture coordinates, assuming that u-coordinates 0.0 and 1.0 share the same location. The textured pillar looks like the following illustration.



If texture wrapping isn't enabled, the rasterizer does not interpolate in the direction needed to generate a believable, reflected image. Rather, the area at the front of the pillar contains a horizontally compressed version of the texels between u-coordinates 0.175 and 0.875, as they pass through the center of the texture. The wrap effect is ruined.

Do not confuse texture wrapping with the similarly named texture addressing modes. Texture wrapping is performed before texture addressing. Be sure the texture wrapping data does not contain any texture coordinates outside of the range of [0.0, 1.0] because this will produce undefined results. For more information about texture addressing, see [Texture addressing modes](#).

Displacement map wrapping

Displacement maps are interpolated by the tessellation engine. Because the wrap mode cannot be specified for the tessellation engine, texture wrapping cannot be performed with displacement maps. An application is able to use a set of vertices that forces the interpolation to wrap in any direction. The application can also specify the interpolation to be done as a simple linear interpolation.

Related topics

[Textures](#)

Texture blending

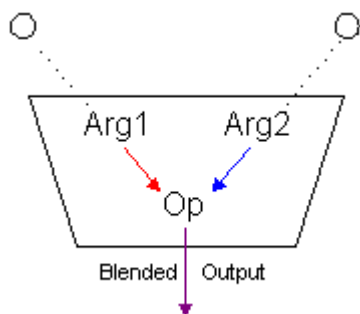
Article • 10/20/2022

Direct3D can blend as many as eight textures onto primitives in a single pass. The use of multiple texture blending can profoundly increase the frame rate of a Direct3D application. An application employs multiple texture blending to apply textures, shadows, specular lighting, diffuse lighting, and other special effects in a single pass.

To use texture blending, your application should first check if the user's hardware supports it.

Texture stages and the texture blending cascade

Direct3D supports single-pass multiple texture blending through the use of texture stages. A texture stage takes two arguments and performs a blending operation on them, passing on the result for further processing or for rasterization. You can visualize a texture stage as shown in the following diagram.

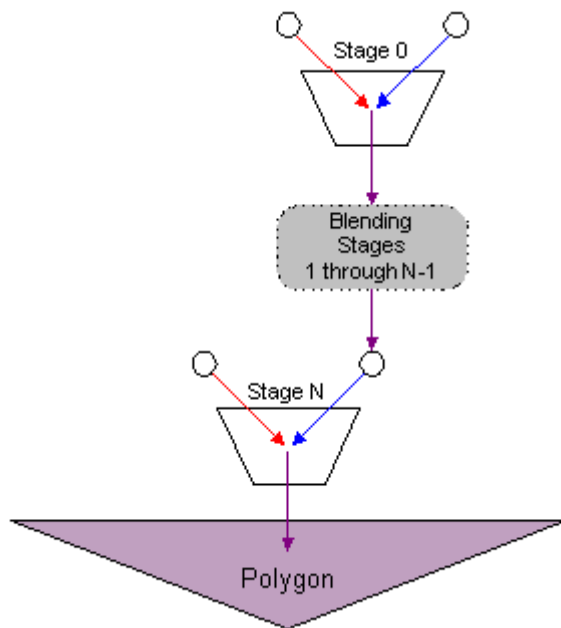


As the preceding diagram shows, texture stages blend two arguments by using a specified operator. Common operations include simple modulation or addition of the color or alpha components of the arguments, but more than two dozen operations are supported. The arguments for a stage can be an associated texture, the iterated color or alpha (iterated during Gouraud shading), arbitrary color and alpha, or the result from the previous texture stage.

Note Direct3D distinguishes color blending from alpha blending. Applications set blending operations and arguments for color and alpha individually, and the results of those settings are independent of one another.

The combination of arguments and operations used by multiple blending stages define a simple flow-based blending language. The results from one stage flow down to

another stage, from that stage to the next, and so on. The concept of results flowing from stage to stage to eventually be rasterized on a polygon is often called the texture blending cascade. The following diagram shows how individual texture stages make up the texture blending cascade.



Each stage in a device has a zero-based index. Direct3D allows up to eight blending stages, although you should always check device capabilities to determine how many stages the current hardware supports. The first blending stage is at index 0, the second is at 1, and so on, up to index 7. The system blends stages in increasing index order.

Use only the number of stages you need; the unused blending stages are disabled by default. So, if your application only uses the first two stages, it need only set operations and arguments for stage 0 and 1. The system blends the two stages, and ignores the disabled stages.

If your application varies the number of stages it uses for different situations - such as four stages for some objects, and only two for others - you don't need to explicitly disable all previously used stages. One option is to disable the color operation for the first unused stage, then all stages with a higher index will not be applied. Another option is to disable texture mapping altogether by setting the color operation for the first texture stage (stage 0) to a disabled state.

In this section

Topic	Description
Blending stages	A blending stage is a set of texture operations and their arguments that define how textures are blended.

Topic	Description
Multipass texture blending	Direct3D applications can achieve numerous special effects by applying various textures to a primitive over the course of multiple rendering passes. The common term for this is <i>multipass texture blending</i> . A typical use for multipass texture blending is to emulate the effects of complex lighting and shading models by applying multiple colors from several different textures. One such application is called <i>light mapping</i> .

Related topics

[Textures](#)

Blending stages

Article • 10/20/2022

A blending stage is a set of texture operations and their arguments that define how textures are blended.

Related topics

[Texture blending](#)

Multipass texture blending

Article • 10/20/2022

Direct3D applications can achieve numerous special effects by applying various textures to a primitive over the course of multiple rendering passes. The common term for this is *multipass texture blending*. A typical use for multipass texture blending is to emulate the effects of complex lighting and shading models by applying multiple colors from several different textures. One such application is called *light mapping*. See [Light mapping with textures](#).

Note Some devices are capable of applying multiple textures to primitives in a single pass. See [Texture blending](#).

If the user's hardware does not support multiple texture blending, your application can use multipass texture blending to achieve the same visual effects. However, the application cannot sustain the frame rates that are possible when using multiple texture blending.

To perform multipass texture blending in a C/C++ application:

1. Set a texture in texture stage 0.
2. Select the desired color and alpha blending arguments and operations. The default settings are well-suited for multipass texture blending.
3. Render the appropriate objects in the scene.
4. Set the next texture in texture stage 0.
5. Set the render states to adjust the source and destination blending factors as needed. The system blends the new textures with the existing pixels in the render-target surface according to these parameters.
6. Repeat Steps 3, 4, and 5 with as many textures as needed.

Related topics

[Texture blending](#)

Light mapping with textures

Article • 10/20/2022

A light map is a texture or group of textures that contains information about lighting in a 3D scene. Light maps map areas of light and shadow onto primitives. Multipass and multiple texture blending enable your application to render scenes with a more realistic appearance than shading techniques.

For an application to realistically render a 3D scene, it must take into account the effect that light sources have on the appearance of the scene. Although techniques such as flat and Gouraud shading are valuable tools in this respect, they can be insufficient for your needs. Direct3D supports multipass and multiple texture blending. These capabilities enable your application to render scenes with a more realistic appearance than scenes rendered with shading techniques alone. By applying one or more light maps, your application can map areas of light and shadow onto its primitives.

A light map is a texture or group of textures that contains information about lighting in a 3D scene. You can store the lighting information in the alpha values of the light map, in the color values, or in both.

If you implement light mapping using multipass texture blending, your application should render the light map onto its primitives on the first pass. It should use a second pass to render the base texture. The exception to this is specular light mapping. In that case, render the base texture first; then add the light map.

Multiple texture blending enables your application to render the light map and the base texture in one pass. If the user's hardware provides for multiple texture blending, your application should take advantage of it when performing light mapping. This significantly improves your application's performance.

Using light maps, a Direct3D application can achieve a variety of lighting effects when it renders primitives. It can map not only monochrome and colored lights in a scene, but it can also add details such as specular highlights and diffuse lighting.

Information on using Direct3D texture blending to perform light mapping is presented in the following topics.

In this section

Topic	Description
-------	-------------

Topic	Description
Monochrome light maps	Monochrome light mapping enables older adapters to perform multipass texture blending, when an older 3D accelerator board doesn't support texture blending using the alpha value of the destination pixel.
Color light maps	A colored light map uses the RGB data in the light map for its lighting information. An application usually renders 3D scenes more realistically if it uses colored light maps.
Specular light maps	When illuminated by a light source, shiny objects that use highly reflective materials receive specular highlights. Sometimes you can get more accurate highlights by applying specular light maps to primitives, rather than using the specular highlights produced by the lighting module.
Diffuse light maps	Matte surfaces have diffuse light reflection. The brightness of diffuse light depends on the distance from the light source and the angle between the surface normal and the light source direction vector. Texture light maps can simulate complex diffuse lighting.

Related topics

[Textures](#)

Monochrome light maps

Article • 10/20/2022

Monochrome light mapping enables older adapters to perform multipass texture blending, when an older 3D accelerator board doesn't support texture blending using the alpha value of the destination pixel.

Some older 3D accelerator boards do not support texture blending using the alpha value of the destination pixel. These adapters also generally do not support multiple texture blending. If your application is running on an adapter such as this, it can use multipass texture blending to perform monochrome light mapping.

To perform monochrome light mapping, an application stores the lighting information in the alpha data of its light map textures. The application uses the texture filtering capabilities of Direct3D to perform a mapping from each pixel in the primitive's image to a corresponding texel in the light map. It sets the source blending factor to the alpha value of the corresponding texel.

Related topics

[Light mapping with textures](#)

Color light maps

Article • 10/20/2022

A colored light map uses the RGB data in the light map for its lighting information. An application usually renders 3D scenes more realistically if it uses colored light maps.

Related topics

[Light mapping with textures](#)

Specular light maps

Article • 10/20/2022

When illuminated by a light source, shiny objects that use highly reflective materials receive specular highlights. Sometimes you can get more accurate highlights by applying specular light maps to primitives, rather than using the specular highlights produced by the lighting module.

To perform specular light mapping, add the specular light map to the primitive's texture, then modulate (multiply the result by) the RGB light map.

Related topics

[Light mapping with textures](#)

Diffuse light maps

Article • 10/20/2022

Matte surfaces have diffuse light reflection. The brightness of diffuse light depends on the distance from the light source and the angle between the surface normal and the light source direction vector. Texture light maps can simulate complex diffuse lighting.

The diffuse lighting effects simulated by lighting calculations produce only general effects. Your application can simulate more complex diffuse lighting with texture light maps.

Related topics

[Light mapping with textures](#)

Compressed texture resources

Article • 10/20/2022

Texture maps are digitized images drawn on three-dimensional shapes to add visual detail. They are mapped into these shapes during rasterization, and the process can consume large amounts of both the system bus and memory. To reduce the amount of memory consumed by textures, Direct3D supports the compression of texture surfaces. Some Direct3D devices support compressed texture surfaces natively. On such devices, when you have created a compressed surface and loaded the data into it, the surface can be used in Direct3D like any other texture surface. Direct3D handles decompression when the texture is mapped to a 3D object.

Storage efficiency and texture compression

All texture compression formats are powers of two. While this does not mean that a texture is necessarily square, it does mean that both x and y are powers of two. For example, if a texture is originally 512 by 128 bytes, the next mipmapping would be 256 by 64 and so on, with each level decreasing by a power of two. At lower levels, where the texture is filtered to 16 by 2 and 8 by 1, there will be wasted bits because the compression block is always a 4 by 4 block of texels. Unused portions of the block are padded.

Although there are wasted bits at the lowest levels, the overall gain is still significant. The worst case is, in theory, a 2K by 1 texture (2^0 power). Here, only a single row of pixels is encoded per block, while the rest of the block is unused.

Mixing formats within a single texture

It is important to note that any single texture must specify that its data is stored as 64 or 128 bits per group of 16 texels. If 64-bit blocks - that is, block compression format BC1 - are used for the texture, it is possible to mix the opaque and 1-bit alpha formats on a per-block basis within the same texture. In other words, the comparison of the unsigned integer magnitude of color_0 and color_1 is performed uniquely for each block of 16 texels.

Once the 128-bit blocks are used, the alpha channel must be specified in either explicit (format BC2) or interpolated mode (format BC3) for the entire texture. As with color, when interpolated mode (format BC3) is selected, then either eight interpolated alphas or six interpolated alphas mode can be used on a block-by-block basis. Again the

magnitude comparison of alpha_0 and alpha_1 is done uniquely on a block-by-block basis.

Direct3D provides services to compress surfaces that are used for texturing 3D models. This section provides information about creating and manipulating the data in a compressed texture surface.

In this section

Topic	Description
Opaque and 1-bit alpha textures	Texture format BC1 is for textures that are opaque or have a single transparent color.
Textures with alpha channels	There are two ways to encode texture maps that exhibit more complex transparency. In each case, a block that describes the transparency precedes the 64-bit block already described. The transparency is either represented as a 4x4 bitmap with 4 bits per pixel (explicit encoding), or with fewer bits and linear interpolation that is analogous to what is used for color encoding.
Block compression	Block compression is a lossy texture-compression technique for reducing texture size and memory footprint, giving a performance increase. A block-compressed texture can be smaller than a texture with 32-bits per color.
Compressed texture formats	This section contains information about the internal organization of compressed texture formats. You do not need these details to use compressed textures, because you can use Direct3D functions for conversion to and from compressed formats. However, this information is useful if you want to operate on compressed surface data directly.

Related topics

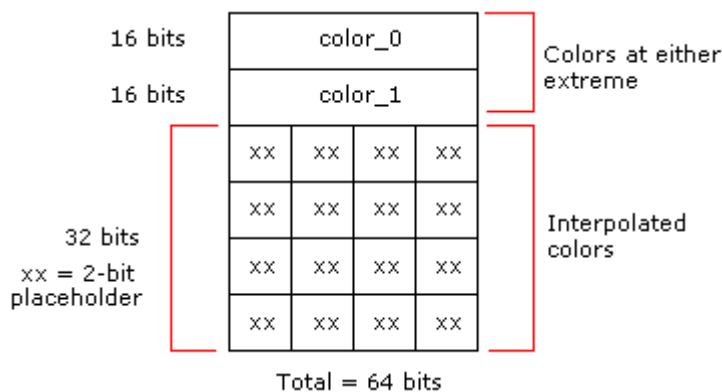
[Textures](#)

Opaque and 1-bit alpha textures

Article • 10/20/2022

Texture format BC1 is for textures that are opaque or have a single transparent color.

For each opaque or 1-bit alpha block, two 16-bit values (RGB 5:6:5 format) and a 4x4 bitmap with 2 bits per pixel are stored. This totals 64 bits for 16 texels, or four bits per texel. In the block bitmap, there are 2 bits per texel to select between the four colors, two of which are stored in the encoded data. The other two colors are derived from these stored colors by linear interpolation. This layout is shown in the following diagram.



The 1-bit alpha format is distinguished from the opaque format by comparing the two 16-bit color values stored in the block. They are treated as unsigned integers. If the first color is greater than the second, it implies that only opaque texels are defined. This means four colors are used to represent the texels. In four-color encoding, there are two derived colors and all four colors are equally distributed in RGB color space. This format is analogous to RGB 5:6:5 format. Otherwise, for 1-bit alpha transparency, three colors are used and the fourth is reserved to represent transparent texels.

In three-color encoding, there is one derived color and the fourth 2-bit code is reserved to indicate a transparent texel (alpha information). This format is analogous to RGBA 5:5:5:1, where the final bit is used for encoding the alpha mask.

The following code example illustrates the algorithm for deciding whether three- or four-color encoding is selected:

C++

```
if (color_0 > color_1)
{
    // Four-color block: derive the other two colors.

    // 00 = color_0, 01 = color_1, 10 = color_2, 11 = color_3
    // These 2-bit codes correspond to the 2-bit fields
    // stored in the 64-bit block.
```

```

    color_2 = (2 * color_0 + color_1 + 1) / 3;
    color_3 = (color_0 + 2 * color_1 + 1) / 3;
}
else
{
    // Three-color block: derive the other color.
    // 00 = color_0, 01 = color_1, 10 = color_2,
    // 11 = transparent.
    // These 2-bit codes correspond to the 2-bit fields
    // stored in the 64-bit block.
    color_2 = (color_0 + color_1) / 2;
    color_3 = transparent;
}

```

It is recommended that you set the RGBA components of the transparency pixel to zero before blending.

The following tables show the memory layout for the 8-byte block. It is assumed that the first index corresponds to the y-coordinate and the second corresponds to the x-coordinate. For example, `Texel[1][2]` refers to the texture map pixel at $(x,y) = (2,1)$.

The following is the memory layout for the 8-byte (64-bit) block:

Word address	16-bit word
0	Color_0
1	Color_1
2	Bitmap Word_0
3	Bitmap Word_1

Color_0 and Color_1, the colors at the two extremes, are laid out as follows:

Bits	Color
4:0 (LSB*)	Blue color component
10:5	Green color component
15:11	Red color component

*least-significant bit

Bitmap Word_0 is laid out as follows:

Bits	Texel
1:0 (LSB)	Texel[0][0]
3:2	Texel[0][1]
5:4	Texel[0][2]
7:6	Texel[0][3]
9:8	Texel[1][0]
11:10	Texel[1][1]
13:12	Texel[1][2]
15:14 (MSB*)	Texel[1][3]

*most significant bit (MSB)

Bitmap Word_1 is laid out as follows:

Bits	Texel
1:0 (LSB)	Texel[2][0]
3:2	Texel[2][1]
5:4	Texel[2][2]
7:6	Texel[2][3]
9:8	Texel[3][0]
11:10	Texel[3][1]
13:12	Texel[3][2]
15:14 (MSB)	Texel[3][3]

Example of opaque color encoding

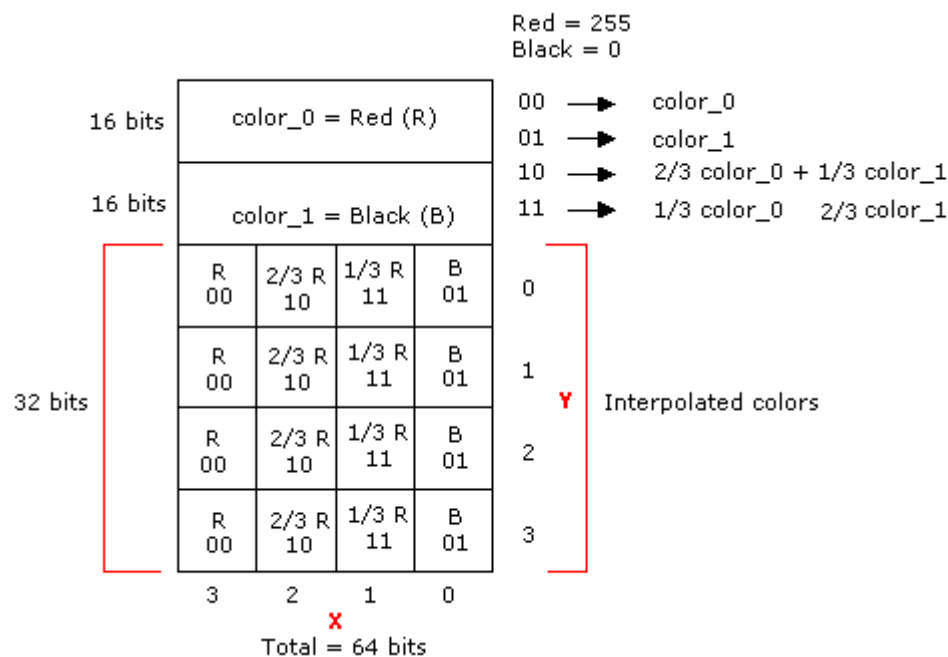
As an example of opaque encoding, assume that the colors red and black are at the extremes. Red is color_0, and black is color_1. There are four interpolated colors that

form the uniformly distributed gradient between them. To determine the values for the 4x4 bitmap, the following calculations are used:

C++

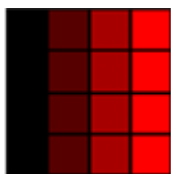
```
00 ? color_0
01 ? color_1
10 ? 2/3 color_0 + 1/3 color_1
11 ? 1/3 color_0 + 2/3 color_1
```

The bitmap then looks like the following diagram.



This looks like the following illustrated series of colors.

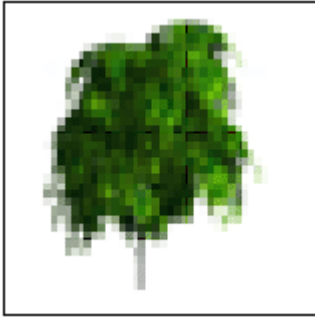
Note In an image, pixel (0,0) appears on the top left.



Example of 1-bit alpha encoding

This format is selected when the unsigned 16-bit integer, color_0, is less than the unsigned 16-bit integer, color_1. An example of where this format can be used is leaves on a tree, shown against a blue sky. Some texels can be marked as transparent while three shades of green are still available for the leaves. Two colors fix the extremes, and the third is an interpolated color.

The following illustration is an example of such a picture.



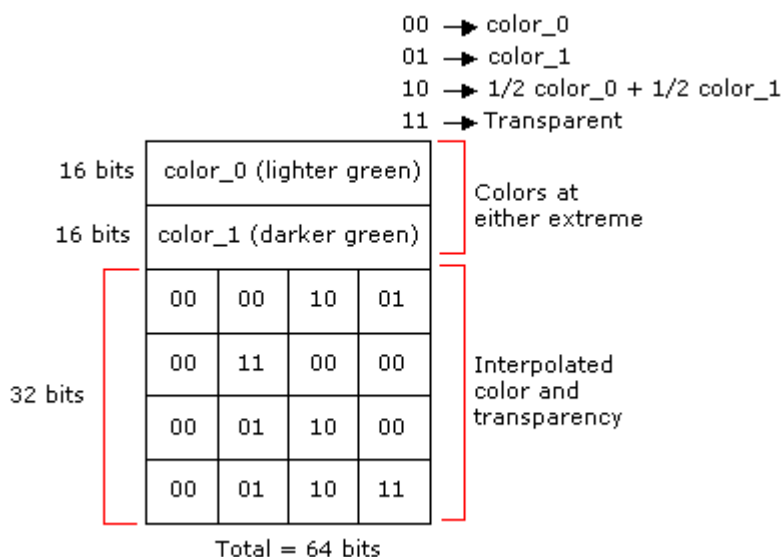
Where the image is shown as white, the texel would be encoded as transparent. The RGBA components of the transparent texels should be set to zero before blending.

The bitmap encoding for the colors and the transparency is determined using the following calculations.

C++

```
00 ? color_0
01 ? color_1
10 ? 1/2 color_0 + 1/2 color_1
11 ? Transparent
```

The bitmap then looks like the following diagram.



Related topics

[Compressed texture resources](#)

Textures with alpha channels

Article • 10/20/2022

There are two ways to encode texture maps that exhibit more complex transparency. In each case, a block that describes the transparency precedes the 64-bit block already described. The transparency is either represented as a 4x4 bitmap with 4 bits per pixel (explicit encoding), or with fewer bits and linear interpolation that is analogous to what is used for color encoding.

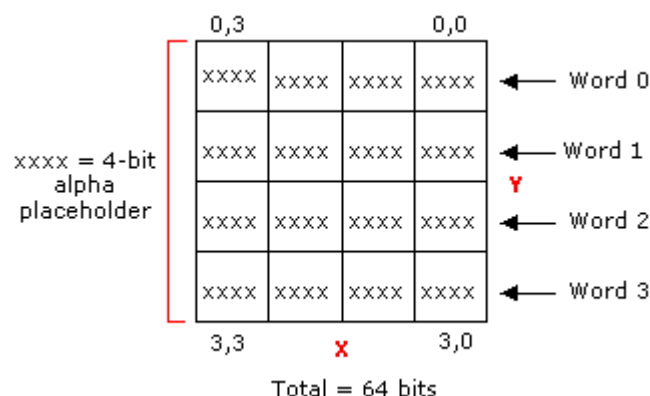
The transparency block and the color block are arranged as shown in the following table.

Word address	64-bit block
3:0	Transparency block
7:4	Previously described 64-bit block

Explicit texture encoding

For explicit texture encoding (BC2 format), the alpha components of the texels that describe transparency are encoded in a 4x4 bitmap with 4 bits per texel. These four bits can be achieved through a variety of means such as dithering or by using the four most significant bits of the alpha data. However they are produced, they are used just as they are, without any form of interpolation.

The following diagram shows a 64-bit transparency block.



Note The compression method of Direct3D uses the four most significant bits.

The following tables illustrate how the alpha information is laid out in memory, for each 16-bit word.

Layout for word 0:

Bits	Alpha
3:0 (LSB*)	[0][0]
7:4	[0][1]
11:8	[0][2]
15:12 (MSB*)	[0][3]

*least-significant bit, most significant bit (MSB)

Layout for word 1:

Bits	Alpha
3:0 (LSB)	[1][0]
7:4	[1][1]
11:8	[1][2]
15:12 (MSB)	[1][3]

Layout for word 2:

Bits	Alpha
3:0 (LSB)	[2][0]
7:4	[2][1]
11:8	[2][2]
15:12 (MSB)	[2][3]

Layout for word 3:

Bits	Alpha
3:0 (LSB)	[3][0]
7:4	[3][1]
11:8	[3][2]
15:12 (MSB)	[3][3]

The color compare used in BC1 to determine if the texel is transparent is not used in this format. It is assumed that without the color compare the color data is always treated as if in 4-color mode.

Three-bit linear alpha interpolation

The encoding of transparency for the BC3 format is based on a concept similar to the linear encoding used for color. Two 8-bit alpha values and a 4x4 bitmap with three bits per pixel are stored in the first eight bytes of the block. The representative alpha values are used to interpolate intermediate alpha values. Additional information is available in the way the two alpha values are stored. If alpha_0 is greater than alpha_1, then six intermediate alpha values are created by the interpolation. Otherwise, four intermediate alpha values are interpolated between the specified alpha extremes. The two additional implicit alpha values are 0 (fully transparent) and 255 (fully opaque).

The following code example illustrates this algorithm.

C++

```
// 8-alpha or 6-alpha block?
if (alpha_0 > alpha_1) {
    // 8-alpha block: derive the other six alphas.
    // Bit code 000 = alpha_0, 001 = alpha_1, others are interpolated.
    alpha_2 = (6 * alpha_0 + 1 * alpha_1 + 3) / 7;    // bit code 010
    alpha_3 = (5 * alpha_0 + 2 * alpha_1 + 3) / 7;    // bit code 011
    alpha_4 = (4 * alpha_0 + 3 * alpha_1 + 3) / 7;    // bit code 100
    alpha_5 = (3 * alpha_0 + 4 * alpha_1 + 3) / 7;    // bit code 101
    alpha_6 = (2 * alpha_0 + 5 * alpha_1 + 3) / 7;    // bit code 110
    alpha_7 = (1 * alpha_0 + 6 * alpha_1 + 3) / 7;    // bit code 111
}
else {
    // 6-alpha block.
    // Bit code 000 = alpha_0, 001 = alpha_1, others are interpolated.
    alpha_2 = (4 * alpha_0 + 1 * alpha_1 + 2) / 5;    // Bit code 010
    alpha_3 = (3 * alpha_0 + 2 * alpha_1 + 2) / 5;    // Bit code 011
    alpha_4 = (2 * alpha_0 + 3 * alpha_1 + 2) / 5;    // Bit code 100
```

```

alpha_5 = (1 * alpha_0 + 4 * alpha_1 + 2) / 5;    // Bit code 101
alpha_6 = 0;                                       // Bit code 110
alpha_7 = 255;                                     // Bit code 111
}

```

The memory layout of the alpha block is as follows:

Byte	Alpha
0	Alpha_0
1	Alpha_1
2	[0][2] (2 MSBs), [0][1], [0][0]
3	[1][1] (1 MSB), [1][0], [0][3], [0][2] (1 LSB)
4	[1][3], [1][2], [1][1] (2 LSBs)
5	[2][2] (2 MSBs), [2][1], [2][0]
6	[3][1] (1 MSB), [3][0], [2][3], [2][2] (1 LSB)
7	[3][3], [3][2], [3][1] (2 LSBs)

The color compare used in BC1 to determine if the texel is transparent is not used with these formats. It is assumed that without the color compare the color data is always treated as if in 4-color mode.

Related topics

[Compressed texture resources](#)

Block compression

Article • 01/18/2023

Block compression is a lossy texture-compression technique for reducing texture size and memory footprint, giving a performance increase. A block-compressed texture can be smaller than a texture with 32-bits per color.

Block compression is a texture compression technique for reducing texture size. When compared to a texture with 32-bits per color, a block-compressed texture can be up to 75 percent smaller. Applications usually see a performance increase when using block compression because of the smaller memory footprint.

While lossy, block compression works well and is recommended for all textures that get transformed and filtered by the pipeline. Textures that are directly mapped to the screen (UI elements like icons and text) are not good choices for compression because artifacts are more noticeable.

A block-compressed texture must be created as a multiple of size 4 in all dimensions and cannot be used as an output of the pipeline.

How block compression works

Block compression is a technique for reducing the amount of memory required to store color data. By storing some colors in their original size, and other colors using an encoding scheme, you can dramatically reduce the amount of memory required to store the image. Since the hardware automatically decodes compressed data, there is no performance penalty for using compressed textures.

To see how compression works, look at the following two examples. The first example describes the amount of memory used when storing uncompressed data; the second example describes the amount of memory used when storing compressed data.

- [Storing uncompressed data](#)
- [Storing compressed data](#)

Storing uncompressed data

The following illustration represents an uncompressed 4×4 texture. Assume that each color contains a single color component (red for instance) and is stored in one byte of memory.

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

The uncompressed data is laid out in memory sequentially and requires 16 bytes, as shown in the following illustration.

1	2	3	...	16
---	---	---	-----	----

Storing compressed data

Now that you've seen how much memory an uncompressed image uses, take a look at how much memory a compressed image saves. The [BC4](#) compression format stores 2 colors (1 byte each) and 16 3-bit indices (48 bits, or 6 bytes) that are used to interpolate the original colors in the texture, as shown in the following illustration.

1	2	16 3-bit encoded colors
---	---	-------------------------

The total space required to store the compressed data is 8 bytes which is a 50-percent memory savings over the uncompressed example. The savings are even larger when more than one color component is used.

The substantial memory savings provided by block compression can lead to an increase in performance. This performance comes at the cost of image quality (due to color interpolation); however, the lower quality is often not noticeable.

The next section shows how Direct3D enables using block compression in an application.

Using block compression

Create a block-compressed texture just like an uncompressed texture except that you specify a block-compressed format.

Next, create a view to bind the texture to the pipeline. Since a block-compressed texture can be used only as an input to a shader-stage, you want to create a shader-resource view.